

From research to execution

Two connected projects. One traceable research loop.

Discussion draft for Ben, Alex and Yaroslav · 8 September 2026 · Draft 0.1

Prepared by Atlas (AI assistant) from Ben's project discussion and public sources. Not yet reviewed or agreed by Alex or Yaroslav. Project names below are descriptive working labels, not final brands.

1. The proposal in plain English

Build two useful projects that cooperate without becoming the same codebase:

- **Project A — Research Laboratory:** a collaborative workspace for turning questions about markets into sourced, reproducible experiments and evaluated portfolio strategies. Ben's proposed starting point is a separate product using Qlib.
- **Project B — Execution Pipeline:** Alex's project, connected through a small adapter if its existing architecture supports that. Its proposed role is to carry out selected portfolio decisions in a simulator, later potentially a paper venue, and keep an accurate record of what happened.
- **The connection:** reviewed portfolio targets go out; orders, fills, costs and reconciled positions come back. Research can then investigate the gap between expectations and outcomes.

Research asks: What should we test, and what does the evidence support?

Execution asks: How do we achieve the selected portfolio from the current account state, including when things go wrong?

Together: Why did the result differ from the research assumptions?

What this document is — and is not

This is a proposal for discussion and independent research. It describes intended responsibilities, not a verified feature list. Alex's repository and implemented lifecycle have not been inspected for this pack. Yaroslav may contribute across both projects; his interests and availability remain to be agreed.

The first scope is historical research plus local simulated execution. Broker-paper work is a later decision. Live-money trading is outside this proposal. No profitable strategy, completed integration or execution-reliability result is claimed.

The recommended first result

One reproducible research run, one portfolio target passed into a recoverable simulator, and one report explaining any difference between the intended and achieved portfolio. A useful negative result counts: discovering a cost assumption or recovery bug is progress.

2. Project A — the research laboratory

Purpose: help a team investigate market hypotheses with provenance, comparable experiments and evidence they can revisit—not merely collect charts or AI-generated trade opinions.

What a researcher would do

1. Write a question, a proposed mechanism and what would falsify it.
2. Select permitted data and record when each observation became available.
3. Build a simple baseline and one candidate feature or model.
4. Run a chronological evaluation with realistic costs and protected test data.
5. Compare results, inspect failures and retain rejected ideas as well as successful ones.
6. Review the strategy before exporting a time-specific portfolio target.

A possible question, not a selected strategy: **does a timestamped feature extracted from company disclosures improve a price-only baseline after costs?** This would require reliable publication times, data rights and a common market. A simple price-based baseline is enough for the first integration; document extraction should not block it.

What Qlib supplies; what the team would own

Qlib provides a machine-learning workflow spanning data processing, model training and backtesting.^[1] It also includes portfolio strategies: its documented weight-based strategy can derive order lists from target positions.^[2] The project should reuse these capabilities rather than rebuild them under a new name.

The proposed product layer is the team's research workspace: hypothesis records, source-linked document features, experiment specifications, shared comparison/review, original research methods and the execution-feedback connection. Initially wrap Qlib behind a small adapter; contribute generally useful improvements upstream where appropriate. Do not fork it without a concrete blocker.

Where AI helps

AI could find and summarise sources, propose testable hypotheses, draft feature code in an isolated workspace, inspect failed experiments and help explain comparisons. Every material claim should link to a source or computed result. Generated code needs review and tests; generated features need leakage checks.

AI-assisted factor/model research already has close reference projects, including RD-Agent.^[8] The proposed value must therefore be demonstrated through reproducibility, collaboration or research usefulness—not the label “AI agents”.

First scope: one market, one horizon, one baseline and one candidate. Defer a large dashboard, autonomous strategy promotion and training a proprietary foundation model.

3. Project B — the execution pipeline

Purpose: turn an approved desired portfolio into controlled execution, durable records and reconciled account state. This is the proposed integration role for Alex's project, not a claim that all these components are already present.

What it would own

- Current positions, available cash, outstanding orders and reserved exposure.
- Instrument identifiers, lot/tick rules and executable price conventions.
- Changes required to move from actual state toward the requested target.
- Independent permissions, freshness checks and operational risk limits.
- Simulator or approved paper-venue interaction, including acknowledgements, partial fills, rejects and cancellations.
- Persistence, restart recovery and reconciliation against authoritative venue reports.

Research determines the desired portfolio. Execution determines how to approach it from the account state that actually exists. A historical order list should not be copied blindly into an account with different holdings or pending orders.

A concrete failure to design around

The venue accepts an order, but the local process loses the acknowledgement. After restart, the system cannot conclude that the order failed. It should inspect recorded intent and venue state, then resolve the uncertainty rather than submit a duplicate blindly.

Keep these meanings separate: **intent accepted**, **order acknowledged**, **partially filled**, **fully filled**, **cancellation requested**, **cancellation confirmed**, and **outcome unknown**. Stopping the process does not remove existing orders or positions.

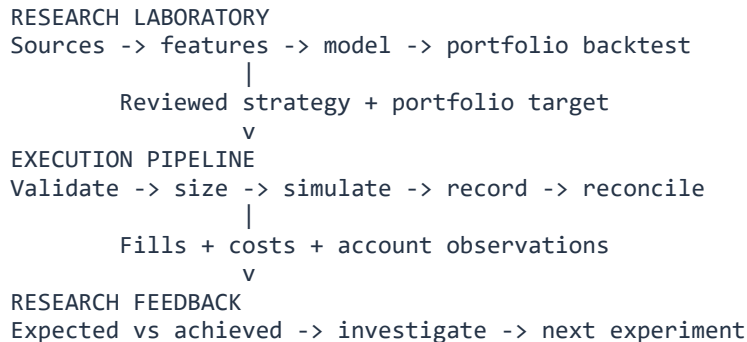
Reuse before reinvention

NautilusTrader documents event-driven execution components and reconciliation, making it a strong architecture reference.^{[3][4]} LEAN's reality models cover matters such as fills, slippage and brokerage assumptions.^[5] Those capabilities are not new inventions of this proposal.

Alex can retain a custom core where it serves a deliberate engineering goal and its verification cost is justified. Otherwise, reuse suitable components. Reading several engines does not mean installing all of them or rewriting his project.

First scope: one isolated simulated account and a bounded order lifecycle. The language, runtime, market and adapter should follow inspection of Alex's actual code.

4. How the two projects connect



Four proposed records

StrategyBundle: identifies the strategy being evaluated: dataset, features, model, portfolio policy, universe, validation report and cost assumptions. It identifies artifacts; it is not permission to load arbitrary generated code into execution.

PortfolioIntent: a time-specific, complete target-weight snapshot. It references the bundle and carries an idempotency key, account/environment, data cutoff, decision time, expiry, execution window and constraints. Cash and omitted-instrument meanings must be explicit.

ExecutionEvent: an append-only observation linked to the intent and order: accepted, rejected, acknowledged, filled, cancelled or unresolved. Record source time and local receipt time separately.

ReconciliationSnapshot: positions, cash and open orders at a stated time, including whether internal records agree with the authoritative execution environment.

Boundaries that matter

- Research holds no broker credentials. Execution checks its own approved strategy/environment/limit record; a request cannot grant itself authority by setting an approval flag.
- Duplicate delivery of an identical intent returns the recorded outcome. The same ID with changed content is rejected.
- Execution does not rewrite historical research results. New observations and corrections are appended and linked.
- Historical simulation uses a historical clock. Re-exporting old predictions today does not make them fresh paper-trading signals.
- Risk constraints from research cannot loosen execution's independent limits. If the target cannot be achieved within the permitted envelope, report the constraint or abstain.

Start simple: local JSON requests and JSON Lines event exports, or Alex's existing suitable API. A file export is not a replacement for durable execution storage. Separate projects do not require microservices, shared databases or a low-latency binary protocol on day one.

The detailed meanings and failure tests are in `02-architecture-and-contracts.md`.

5. How Ben, Alex and Yaroslav could collaborate

These are proposed areas of ownership, not assigned jobs or commitments. The group should choose based on interest, existing work and available time.

Ben — proposed research-side coordination

Develop the research workflow, hypothesis/experiment format, Qlib integration, baseline evaluation and the product experience for investigating results. Work with Alex on what a reviewed strategy exports and with Yaroslav on data quality and reproducibility.

Alex — proposed execution-side coordination

Map the existing pipeline, select the smallest integration adapter, and coordinate execution-state ownership, order lifecycle, operational limits and recovery. Work with the research side on assumptions that affect achievable portfolios and costs.

Yaroslav — a contributor across both projects

Useful cross-project work could include:

- **Data and timing integrity:** instrument mappings, calendars, corporate-action conventions, data cutoffs and leakage tests.
- **Integration and reproducibility:** a portable contract-test corpus, deterministic simulator scenarios and end-to-end replay.
- **Analysis and product work:** comparing targets with achieved holdings, investigating costs and presenting evidence clearly.
- **Research or systems features:** a substantive feature/model experiment or execution component, according to his interests—not only support or testing work.

A strong first option is to help trace one research decision through both systems and own a bounded part of the comparison or test suite. This gives him architectural context on both sides without making him responsible for every gap.

Working agreements

Keep each project independently useful. Choose one canonical owner for shared contract files, with review from both consumers. Avoid parallel edits to the same shared file. Record short decisions with rationale and an owner; disagreements become questions to test, not assumptions silently embedded in code.

If someone authors a change, another person should review its evidence before integration. Review is about the real artifact and tests, not whose agent sounds most confident. Disclose materially AI-generated work and keep human approval separate from agent output.

No deadlines, repository ownership transfers, licences or contribution quotas are decided by this document.

6. A staged first build, with proof at each step

M0 — Inspect and agree

Trace one actual input-to-order-to-state path in Alex's repository. Establish the research baseline and choose a compatible market, horizon and account profile. Decide what to reuse and who owns the boundary. **Proof:** a source-linked capability map and an agreed small change list.

M1 — Freeze a research reference

Complete one Qlib portfolio backtest with identifiable data, model, policy, starting account state and cost assumptions. Export its targets. **Proof:** a rerun reproduces declared outputs within stated tolerances. Synthetic lifecycle fixtures remain separate from market-performance evidence.

M2 — Complete the simulator round-trip

Export one target, validate it in execution, simulate the required changes, and import events and a reconciled snapshot into the research run. **Proof:** a trace from research decision to achieved holdings. Differing fill assumptions should produce an explained difference, not forced identical results.

M3 — Prove bounded recovery behavior

Exercise duplicate intents, changed-payload ID reuse, restart, lost acknowledgement, partial-fill/cancel races, duplicate reports, expired requests and state mismatches. **Proof:** repeatable traces with state invariants. If evidence cannot resolve an external outcome, abstaining is correct; arbitrary brokers do not acquire exactly-once guarantees through a local test.

M4 — Explain the gap

Show target versus achieved holdings, fees, delays, rejects, unfilled quantities and implementation shortfall against a stated price benchmark. **Proof:** seeded faults appear in an event-linked comparison report. Counterfactual replay is labelled as simulation, not an observed or uniquely proven cause.

Stop here for the first integrated milestone: a reproducible research run, a recoverable simulator and a useful discrepancy report.

Later — optional broker-paper integration

Only after review and explicit agreement on the venue/account setup. Use current data and compatible instruments, verify that venue's paper semantics, and reconcile its reports. Paper fills remain simulated; they do not establish live-market realism or profitable predictive edge.

Keep success measures separate

- Research usefulness: can another person reproduce and understand the experiment?
- Prediction: does a candidate beat its declared baseline on protected chronological data?
- Portfolio evaluation: what survives costs, turnover, drawdown and sensitivity tests?
- Execution: do lifecycle and recovery invariants hold under specified faults?

7. Where the interesting engineering and research could be

First: explain the research-to-execution gap

Build a discrepancy investigator that connects a hypothesis and dataset cutoff to targets, order attempts, fills and final positions. Ask: **where did expectation and outcome first diverge?**

This gives the two projects a practical reason to cooperate. Its contribution is a usable cross-project evidence trail—not simply another PnL chart. Avoid claiming unique causation when market effects and operational events overlap.

Second: make failures reproducible

A controllable fake venue, virtual clock and portable test corpus could test lost acknowledgements, message duplication and cancellation races across supported adapters. Minimise failures into regression traces. Reuse an existing harness if one fits; the valuable result is demonstrated invariants for the chosen scope, not novelty by assertion.

Later: model execution uncertainty

With suitable observations, investigate fill probability, slippage, delay and unfilled opportunity cost. Start with fixed or simple empirical baselines before learned models. Use chronological holdouts and calibration checks; keep unfilled and rejected attempts rather than studying successful fills alone.

HftBacktest is a useful specialist reference: it separates feed, order-entry and order-response latency, and documents queue/fill models.[\[6\]\[7\]](#) Its replay cannot change the historical market, so market impact and some liquidity-taking fills are limitations, not validated reality.[\[6\]](#) Daily bar data alone is not evidence of queue position. Paper-generated data calibrates that paper environment, not automatically a live venue.

Read these projects with a question

- **Qlib**: trace data to prediction to portfolio strategy. What can be reused without modifying the engine?[\[1\]\[2\]](#)
- **RD-Agent**: investigate its automated R&D/factor-model approach. What would the team's workflow add or deliberately avoid?[\[8\]](#)
- **NautilusTrader**: trace one order and a lost-acknowledgement recovery path.[\[3\]\[4\]](#)
- **LEAN**: identify the broker/fill/slippage assumptions that change simulated results.[\[5\]](#)
- **HftBacktest**: vary queue/latency assumptions and identify what replay cannot establish.[\[6\]\[7\]](#)

An AI investigation assistant could cite events, propose explanations and draft replay tests after the evidence layer works. It should not approve its own strategy, alter limits or operate account credentials.

These are hypotheses about useful product work, not an exhaustive market survey or proof that nobody else has built it.

8. Questions for the group and its agents

Decisions worth making together

1. What does Alex's pipeline actually implement today, and what part does he want to keep building?
2. Which single market, instrument universe and decision horizon fit both projects?
3. What research question would make the laboratory useful to the group first?
4. Would the first boundary carry target weights, quantities or orders? Target weights are the proposed default; challenge them against the real pipeline.
5. Which components should be reused, and where is a custom implementation genuinely worthwhile?
6. Which part does Yaroslav want to own, and where would he prefer to collaborate?
7. Who maintains the shared contract, and what evidence is needed before accepting a change?
8. What observation would cause the group to simplify, change direction or stop a workstream?

How to use an agent on this pack

Give it the Markdown files and ask for an independent critique, not implementation. The included `04-agent-research-brief.md` provides bounded research tracks, a copy-ready prompt and an evidence format. Ask it to inspect primary sources and distinguish **documented**, **locally verified**, **proposed**, **inferred** and **unknown** claims.

A useful response challenges scope, identifies existing solutions, tests whether the boundaries make sense, and proposes the smallest informative experiment. “Looks great” without source or failure analysis is not a review.

Small glossary

- **Backtest:** applying a strategy to historical data under stated simulation assumptions.
- **Point-in-time:** using only information actually available at the decision time, not merely records with an old event date.
- **Target / intent:** the desired portfolio; not proof that orders were accepted or filled.
- **Fill:** an execution of some or all of an order.
- **Idempotency:** repeated delivery of the same request does not repeat its intended economic effect.
- **Reconciliation:** comparing internal state with authoritative external/simulator records and resolving discrepancies.
- **Paper execution:** execution through a venue's simulated account, distinct from both historical backtesting and live-money trading.
- **Implementation shortfall:** execution outcome measured against an explicitly chosen reference-price benchmark, including relevant costs and unfilled opportunity cost.

Next conversation: Alex brings a repository/branch or diagram and sample messages; Ben brings a small research run; Yaroslav chooses a contribution he wants to explore. The group agrees one compatible slice before anyone starts a broad rewrite.

Sources

Public sources retrieved 8 September 2026. These support the upstream capabilities described, not a verified implementation of either proposed project. Documentation can change; pin versions and source commits when making implementation decisions.

[1] <https://github.com/microsoft/qlib> — Qlib overview

[2] <https://qlib.readthedocs.io/en/latest/component/strategy.html> — Qlib portfolio strategy

[3] <https://nautilustrader.io/docs/latest/concepts/architecture> — NautilusTrader architecture

[4] <https://nautilustrader.io/docs/latest/concepts/live> — NautilusTrader live lifecycle

[5] <https://www.quantconnect.com/docs/v2/writing-algorithms/reality-modeling/key-concepts> — LEAN reality modelling

[6] https://hftbacktest.readthedocs.io/en/latest/order_fill.html — HftBacktest order fills

[7] https://hftbacktest.readthedocs.io/en/latest/latency_models.html — HftBacktest latency

[8] <https://github.com/microsoft/RD-Agent> — RD-Agent overview